

SUBQUERY

Query inside another query.

~~Query Step 4~~

Step 4 → Aggregation → Main Query

Step 3 → Transformation → Sub query

Step 2 → Filtering

→ Sub query

Step 1 → Join Table

→ Sub query

Inner.

Dependency

Non-correlated
Sub-Query

Correlated
Query.

Result Types

Scalar
Subquery

ROW
Sub Query

Table
Sub Query.

Location / clauses

SELECT

FROM

JOIN

WHERE

comparison
operator

<, >, =, !=, >=, <=

logical
operator

IN, ANY, ALL,
EXIST.

Result types :

Scalar Subquery : Single value. e.g. $AVG(sales)$

Row Subquery : Multiple rows & columns.

1
2
3
4

SELECT

custid

FROM sales_order

Table Subquery : Multiple rows & columns.

1	A
2	B
C	C

Location / Clause :

~~FROM clause~~

FROM clause : uses a temporary table for the main query.

Syntax. SELECT col1, col2, ... → Main Query.

FROM (SELECT col FROM Table WHERE condn) AS alias

Ex:- SELECT

FROM

SELECT

pid,

price,

$AVG(price) OVER avgp$

FROM sales_order

) +

where price > avgp

Ex:- SELECT

RANK() OVER (ORDER BY TS DESC) CR

FROM

SELECT

custid,

$SUM(sales) TotalSales$

FROM sales_order

GROUP BY custid) +

SELECT clause : used to aggregate data side by side with main query's data, allowing for direct comparison.

Syntax. SELECT $\longrightarrow$ Main Query
column1,

(SELECT column FROM table1 WHERE condi) AS alias
FROM Table1
Subquery

Rule : only scalar subqueries are allowed to be used. (only single value).

Ex :
SELECT
pid,
product,
price,
(SELECT
COUNT(*) ~~from orders~~
FROM sales.orders) AS TotalOrders
FROM sales.products.

JOIN clause : used to prepare the data (filtering or aggregation) before joining it with other tables.

Ex :
SELECT
c.c,
o.to
FROM sales.customer c
LEFT JOIN (
SELECT
custid,
COUNT(*) TotalOrders
FROM sales.orders
GROUP BY custid) o
ON c.custid = o.custid.

WHERE Subquery : used for complex logic and makes query more flexible and dynamic.

Comparison operators: use to filter data by comparing two values.

Syntax SELECT col1, col2 → Main Query.
FROM Table1
WHERE column = (SELECT col FROM Table2
WHERE condn)
Subquery

Rule: only scalar subqueries are allowed.
only single value.

Ex: SELECT
FROM sales.products
WHERE price > (
SELECT
AVG(price) avg_price
FROM sales.products
)

IN OPERATOR: ~~check~~ check whether a value matches any value from a list.

Syntax SELECT col1, col2 → Main Query.
FROM Table1
WHERE col IN (SELECT col FROM table2 WHERE condn)
Subquery

Ex: SELECT *
FROM sales.orders
WHERE customer_id IN (
SELECT
customer_id
FROM sales.customers
WHERE country = 'Germany')
USE NOT IN to get all order who are not from Germany.

ANY/ALL :: Check if a value matches all values within a list.

~~Check~~ check if a value matches ANY value within a list.
used to check if a value is true for AT LEAST one of values in a list.

Syntax. SELECT col1, col2 ... → Main Query.
FROM table 1

WHERE col < ANY/ALL (SELECT col FROM table 1 WHERE condn)

Ex: ~~SELECT~~ (saved in laptop)
~~File~~ File.
~~File~~ File.

* ~~NON-CORRELATED~~ CORRELATED SUBQUERY: Subquery that can run independently from Main Query.

* CORRELATED SUBQUERY: Subquery that relies on values from Main Query.

Ex: ~~Query in laptop~~ ~~Query in laptop~~ SELECT

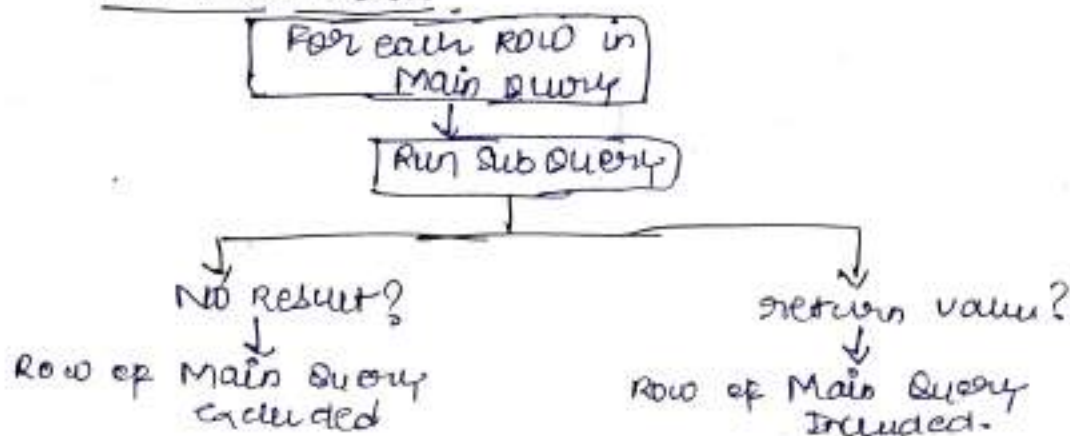
SELECT COUNT(*) FROM sales.orders o WHERE o.custid = c.custid) T5
FROM sales.custome c.

CORRELATED SUBQUERY (EXIST): checks if a subquery returns any rows.

Syntax SELECT col1, col2 --
FROM table 2

WHERE EXISTS (SELECT 1
FROM table 1
WHERE Table 1.ID = table 2.ID)

How EXIST work



Ex:
 SELECT
 *
 FROM sales.orders O
 WHERE BAISTS (SELECT

~~customers~~ 1

FROM sales.customers C
 WHERE Country = 'Germany'
 AND C.customer ID = O.customer ID)

✓ CTE

Temporarily, named result set (Virtual Table)
 that can be multiple times within your query to
 simplify and organize complex query.

~~Another instance~~

~~with query~~

CTE TYPES

Non-Recursive
 CTE

Recursive
 CTE

Standalone
 CTE

Nested
 CTE

standalone CTE. Define and used independently.
 Runs independently as it's self-contained
 and doesn't rely on other CTEs or queries.



Syntax

WITH CTE-NAME AS

```
{  
  SELECT --  
  FROM --  
  WHERE --  
}
```

← CTE Query.

```
SELECT --  
FROM CTE-NAME  
WHERE --
```

→ Main Query.

Ex:- step 1

```
SELECT  
  custid  
  SUM(salus) Totalsalus  
FROM Salus.0  
Group By custid
```

Ex:- Example is saved in laptop.

Multiple Standalone CTE

Syntax

WITH CTE-name1 AS

```
{  
  SELECT  
  FROM  
  WHERE  
}
```

CTE-name2 AS

```
{  
  SELECT  
  FROM  
  WHERE  
}
```

```
SELECT  
FROM CTE-name1  
JOIN CTE-name2  
WHERE
```

Ex:- saved in laptop.

Nested CTE

CTE inside another CTE.

A nested CTE uses the result of another CTE, so it can't run independently.

Syntax

WITH CTE-name1 AS

```
{  
  SELECT --  
  FROM --  
  WHERE --  
}
```

Standalone
CTE

Nested CTE

```

1 CTE_name2 AS
(
  SELECT
  FROM CTE_name 1
  WHERE
  )
SELECT
FROM CTE_name 2
WHERE --
  
```

Ex:- saved in laptop.

Recursive CTE :- self-referencing query that repeatedly processes data until a specific condition is met.

Syntax :

```

WITH CTE_Name AS
(
  SELECT ---
  FROM ---
  WHERE --
  UNION ALL
  SELECT
  FROM CTE_Name
  WHERE (Break condition)
)
  
```

Anchor Query.

Recursive Query.

Use union all bcoz in one query there is no 2 select.

-- Main Query

```

Main {
  SELECT
  FROM CTE_Name
  WHERE --
}
  
```

MA RECURSION! increase the recursion.

Ex:- save in laptop.

Views

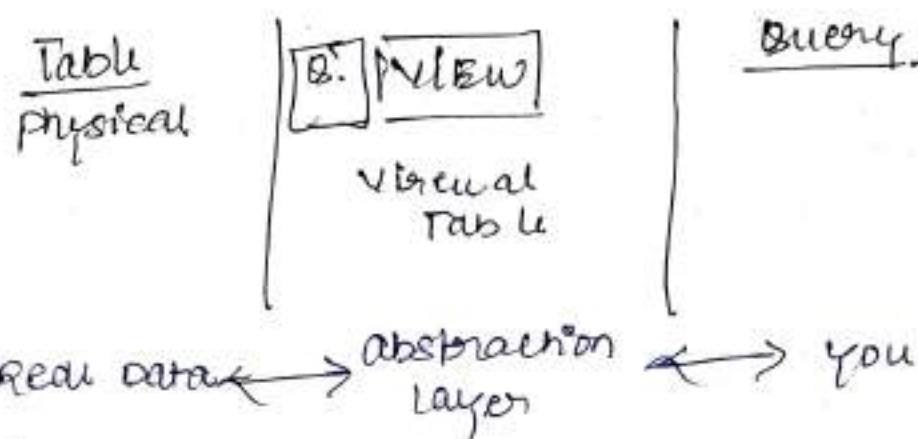
- Databases server : stores, manages and provides access to databases for users or application.
- Database : collection of information that is stored in a structured way.
- Schema : logical layer that groups related objects together.
- Table : place where data is stored and organized into rows and columns.
- VIEW : Virtual table that shows data without storing it physically.
- DDL : set of commands that allow to define and manage the structure of database.

3 tier architecture.

- 1) External (view layer) : Top layer, where end user or applications interact with database.
 - Defines how users view the data.
 - Different users has different views.
- 2) Conceptual (logical layer) : Defines what data is stored in the database and relationships b/w them.
 - Hides physical storage details.
 - Maintains logical structure - tables, columns, data types, & (PK primary / foreign key).
- 3) Internal (level physical) : defines how data is stored in Memory or disk.
 - Deal with indexes, file organization, data blocks, compression etc.
 - Managed by DBMS engine, not visible to users.

View is Virtual Table based on result set of a Query, without storing the data in database.

- Views are persisted SQL queries in the database.



~~Diagram~~ # 1 use case

Central Query logic: Store central, complex query logic in the database for access by multiple queries, reducing project complexity.

VIEW	CTE
• Reduce Redundancy in Multi-Queries. • Improve Reusability in Multi-Queries. • persisted logic • Need to Maintain - create/drop	• Reduce Redundancy in 1 Query. • Improve Reusability in 1 Query. • Temporary logic • No maintenance - auto cleanup.

VIEW (Syntax)

DDL Command $\leftarrow$ CREATE VIEW VIEW_NAME AS
(
 SELECT ---
 FROM ---
 WHERE ---
)

Query $\rightarrow$

NOTE:- If a table or view is created without specifying a Schema, it default to the DBD.

Delete View

DROP VIEW VIEW_NAME

UPDATE (In SQL server)

1) Drop the view.

2) Now create a view.

T-SQL : Transaction SQL is an extension of SQL that adds programming features.

* How DB Execute Views

USE CASE (1) : HIDE COMPLEXITY : Views can be use to hide the complexity of a database tables and offers users more friendly and easy to consume objects.

USE CASE (2) : Data Security : use views to enhance security and protect sensitive data, by hiding columns and/or rows from tables.

USE CASE (3) : Flexibility and dynamic :

USE CASE (4) : Multiple Languages :

USE CASE (5) : Virtual Data Marts in DWH : Views can be used as data marts in data warehouse system. Because they provide a flexible and efficient way to present data.